

Laboratório L2: Criptografia Assimétrica

RSA-OAEP, cifração híbrida, assinatura digital e proteção de chaves

Data:	20 de agosto de 2026 (Não serão aceitas entregas atrasadas)
Modalidade:	aula online assíncrona
Duração prevista:	100 minutos
Organização:	atividade individual
Evidência:	Relatório L2 e código-fonte
Pontuação:	Até 10 pontos
Atenção:	Esse laboratório lhe convida a ir além.

Questão norteadora: como combinar confidencialidade, autenticidade e integridade na troca de arquivos sem compartilhar previamente uma chave secreta, e quais garantias ainda dependem da autenticação da chave pública?

1 Objetivos de aprendizagem

Ao final deste laboratório, o(a) estudante deverá ser capaz de:

- gerar, serializar e carregar um par de chaves RSA;
- distinguir as funções e os requisitos de proteção das chaves pública e privada;
- explicar por que RSA não deve ser empregado diretamente para cifrar arquivos extensos;
- implementar cifração híbrida usando RSA-OAEP e AES-256-GCM;
- criar e verificar assinaturas RSA-PSS com SHA-256;
- demonstrar a detecção de adulterações e discutir o problema da autenticidade da chave pública.

Esse laboratório lhe convida a ir além, então existem menções a ferramentas que você precisará buscar informações extras de forma curiosa sobre elas para complementar sua formação.

2 Fundamentação

Na criptografia assimétrica, cada participante possui um par de chaves relacionadas: uma chave pública, que pode ser distribuída, e uma chave privada, que deve permanecer sob controle exclusivo do titular. Para confidencialidade, o remetente cifra um segredo usando a chave pública do destinatário; somente a chave privada correspondente permite recuperá-lo. Para assinatura, o titular assina com sua chave privada e terceiros verificam com a chave pública.

Neste laboratório, será utilizada uma construção híbrida:

1. gerar uma chave de sessão aleatória K de 256 bits;
2. cifrar o arquivo M com AES-GCM: $(C, T) = \text{AES-GCM}_K(M, N)$;
3. cifrar K com a chave pública RSA do destinatário usando OAEP;
4. armazenar a chave encapsulada, o *nonce*, o texto cifrado e a etiqueta de autenticação;
5. recuperar K com a chave privada e então decifrar e autenticar o arquivo.

O OAEP deverá usar SHA-256 tanto no algoritmo principal quanto em MGF1. Para assinaturas, será empregado RSA-PSS com SHA-256. Esses preenchimentos probabilísticos evitam o uso inseguro do RSA “puro” ou determinístico.

Sobre os algoritmos AES-256-GCM, RSA-OAEP e RSA-PPS e a função MGF1, esses quatro termos são padrões de criptografia modernos usados para proteger dados em trânsito e em descanso. Eles dividem-se entre criptografia simétrica (chave única), assimétrica (chave pública/privada) e assinatura digital.

O **AES-256-GCM** (Criptografia Simétrica de Alta Velocidade) é um algoritmo de criptografia simétrica (a mesma chave cifra e decifra) muito utilizado em conexões seguras como TLS/HTTPS e VPNs. O AES-256 usa o algoritmo *Advanced Encryption Standard* com uma chave de 256 bits, garantindo resistência contra ataques de força bruta. O GCM (Galois/Counter Mode) é um modo de operação AEAD (Authenticated Encryption with Associated Data). Ele cifra os dados e gera um rótulo de autenticação. Isso impede que um atacante altere o conteúdo cifrado sem ser detectado.

O **RSA-OAEP** (*Optimal Asymmetric Encryption Padding*) é um esquema seguro para cifrar dados usando o algoritmo RSA. O RSA tradicional puramente matemático é vulnerável a diversos ataques; o esquema OAEP adiciona um preenchimento aleatório (padding) antes do cálculo do RSA. Seu uso principal está em criptografar pequenos volumes de dados, como chaves de sessão simétricas (troca de chaves). Ele torna o cifra probabilística, ou seja, a mesma mensagem cifrada duas vezes resultará em textos cifrados completamente diferentes.

O **RSA-PSS** (*Probabilistic Signature Scheme*) é um esquema moderno e seguro para gerar e verificar assinaturas digitais usando RSA. Seu uso principal visa garantir a autenticidade e o não-repúdio (provar quem assinou um documento ou código). Ele adiciona aleatoriedade ao processo de assinatura, eliminando riscos de vazamento da chave privada por meio de ataques teóricos.

O **MGF1** (*Mask Generation Function 1*) é uma função de geração de máscara usada em algoritmos de criptografia assimétrica baseados em RSA. Ele pega uma entrada de tamanho variável (como um hash) e produz uma saída pseudoaleatória do comprimento que você desejar. O MGF1 é o motor fundamental por trás dos esquemas de preenchimento do RSA.

Atenção

Não implemente manualmente a aritmética RSA nem invente um formato criptográfico próprio para uso real. Neste laboratório, use as primitivas da biblioteca `cryptography`. A montagem do pequeno contêiner didático é permitida somente para compreender quais elementos precisam acompanhar uma mensagem cifrada.

3 Ambiente e materiais

Use Python 3.10 ou superior, localmente ou no Google Colab. Instale a dependência com:

```
1 pip install -r requirements.txt
```

O pacote contém:

- `codigo_base_l2.py`: estrutura inicial com funções incompletas;
- `mensagem_confidencial.json`: arquivo sintético para cifração híbrida;
- `comunicado_assinado.txt`, `comunicado_assinado.sig` e `chave_publica_docente.pem`: conjunto para verificação de uma assinatura válida;
- `dados_binarios.bin`: entrada binária sintética;
- `SHA256SUMS.txt`: resumos dos arquivos de entrada;
- `README.txt`: instruções de preparação e execução.

Todos os dados são sintéticos. Não utilize dados pessoais, credenciais, chaves reais ou documentos confidenciais.

No laboratório, use a biblioteca `cryptography` que será instalada com:

```
pip install cryptography
```

Ou usando o arquivo fornecido:

```
pip install -r requirements.txt
```

Depois, os alunos poderão importá-la no código Python:

```
from cryptography.hazmat.primitives.asymmetric import rsa , padding
from cryptography.hazmat.primitives import hashes , serialization
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
```

Ela oferece as operações usadas no Laboratório L2:

- geração de pares de chaves RSA;
- serialização de chaves em PEM;
- proteção da chave privada com senha;
- cifração RSA-OAEP;
- assinatura RSA-PSS;
- verificação de assinaturas;
- cifração autenticada AES-GCM.

Apesar do nome genérico, `cryptography` é o nome oficial do pacote instalado pelo `pip`. Ela não pertence à biblioteca-padrão do Python, por isso precisa ser instalada separadamente.

4 Atividades práticas

4.1 Parte A – Geração e proteção do par RSA (20 minutos)

Complete as funções indicadas no arquivo-base para:

1. gerar um par RSA de 3072 bits, com expoente público 65537;
2. serializar a chave pública em PEM no formato `SubjectPublicKeyInfo`;
3. serializar a chave privada em PEM no formato PKCS#8, protegida por senha;
4. carregar novamente as duas chaves;
5. exibir a impressão digital SHA-256 da chave pública.

A senha deve ser obtida durante a execução por `getpass.getpass()` e não pode aparecer no código, no relatório ou no arquivo de entrega. Compare os cabeçalhos PEM das duas chaves e registre as permissões recomendadas para o arquivo privado em um sistema multiusuário.

4.2 Parte B – Cifração híbrida de arquivo (35 minutos)

Implemente `cifrar_arquivo_hibrido()` e `decifrar_arquivo_hibrido()` de acordo com os requisitos:

- chave de sessão AES com 32 bytes, gerada por `AESGCM.generate_key(bit_length=256)`;
- *nonce* aleatório de 12 bytes, sem reutilização com a mesma chave;
- AES-GCM para cifrar todo o conteúdo do arquivo;
- RSA-OAEP/SHA-256 para cifrar somente a chave de sessão;
- metadados autenticados (AAD) iguais à sequência `b"L2-CCSC-2026"`;
- saída em JSON, codificando os campos binários com Base64.

O contêiner deverá possuir os campos `algoritmo`, `chave_encapsulada`, `nonce`, `aad` e `cifrado`. Cifre e recupere `mensagem_confidencial.json` e `dados_binarios.bin`. Compare os arquivos originais e recuperados por SHA-256.

Depois, altere um byte do campo `cifrado` e tente decifrar. Registre a exceção observada e explique qual propriedade do AES-GCM foi demonstrada.

4.3 Parte C – Assinatura digital e adulteração (30 minutos)

Implemente `assinar()` e `verificar_assinatura()` usando:

- RSA-PSS;
- MGF1 com SHA-256;

- `PSS.MAX_LENGTH`;
- SHA-256 como função de resumo.

Realize os experimentos:

1. verifique a assinatura de `comunicado_assinado.txt` usando `chave_publica_docente.pem`;
2. altere um caractere em uma cópia do comunicado e confirme que a verificação falha;
3. assine `mensagem_confidencial.json` com sua chave privada;
4. verifique a assinatura com sua chave pública;
5. tente verificar a assinatura com a chave pública fornecida no pacote e interprete o resultado.

Não trate uma assinatura inválida como erro inesperado do programa. Sua função deve retornar `False` ao capturar especificamente `InvalidSignature`.

4.4 Parte D – Análise de segurança (15 minutos)

Responda de forma fundamentada:

1. Por que cifrar diretamente um arquivo inteiro com RSA é inadequado? Considere limite de tamanho, desempenho e preenchimento.
2. O que cada mecanismo oferece: RSA-OAEP, AES-GCM e RSA-PSS?
3. A verificação correta da assinatura prova quem é a pessoa que controla a chave privada? O que falta para associar uma chave pública a uma identidade?
4. O que aconteceria se um atacante substituísse a chave pública do destinatário antes da cifração?
5. Por que a chave privada deve ser protegida em repouso e quais limitações permanecem mesmo quando o PEM usa senha?

5 Relatório L2

Produza um relatório em PDF, com extensão sugerida de 3 a 5 páginas, contendo:

1. identificação do(a) estudante;
2. descrição da implementação e das escolhas criptográficas;
3. impressão digital da chave pública gerada, sem incluir a chave privada ou a senha;
4. evidências da cifração e recuperação dos dois arquivos;
5. evidências dos testes de adulteração e assinatura inválida;

6. respostas da Parte D;
7. conclusão relacionada à questão norteadora;
8. referências consultadas e declaração de uso de IA, se aplicável.

Entrega obrigatória

Entregue `L2_NomeSobrenome.zip`, contendo somente:

- `relatorio_L2.pdf`;
- `codigo_base_l2.py` concluído;
- sua chave pública e as assinaturas produzidas;
- os contêineres cifrados e arquivos recuperados solicitados;
- opcionalmente, `README.txt` com instruções de execução.

Não inclua chave privada, senha, arquivo `.env`, credenciais ou qualquer outro segredo na entrega.

6 Critérios de avaliação

Critério	Peso
Geração, serialização, carregamento e proteção das chaves RSA	20%
Cifração híbrida correta com RSA-OAEP e AES-GCM	30%
Assinatura e verificação corretas com RSA-PSS	25%
Testes negativos e análise das propriedades de segurança	20%
Clareza, organização e reprodutibilidade da entrega	5%

7 Regras acadêmicas e de segurança

- A atividade é individual. Código, experimentos e relatório devem ser autorais.
- Use exclusivamente os arquivos sintéticos fornecidos ou dados criados especificamente para esta atividade.
- Caso utilize IA generativa, registre ferramenta, finalidade e verificações realizadas, conforme as regras da disciplina.
- Não exponha chaves privadas nem tente recuperar ou usar chaves de terceiros.
- Não use algoritmos obsoletos ou configurações não solicitadas, como RSA sem preenchimento, PKCS#1 v1.5 para cifração, SHA-1 ou AES-ECB.